



Lab 3.2. Managing Applications with Argo CD

In this lab, we're moving beyond the basics and getting into the practical implementation of Argo CD, showcasing its robust capabilities in deploying applications into a Kubernetes cluster. This session is tailored to reinforce your understanding of Argo CD by offering a hands-on experience in a real-world setting.

You'll not only deploy an application using Argo CD and GitOps principles but also learn how to modify the deployed application. Witness firsthand how Argo CD swiftly updates pods in response to changes, and understand the process of rolling back to a previously deployed version, ensuring a smooth and reliable deployment experience.

Objective

Learn to deploy an application to Kubernetes using Argo CD and GitOps principles. In this lab, you will deploy an example application.

Prerequisites

- Argo CD already installed and running in your Kubernetes cluster.
 - See Chapter 2's "Deploying Kubernetes for Argo" section for details on how to set up a cluster
- Basic understanding of Kubernetes and Argo CD.
- Git hosting service (GitHub recommended).

Deploy the Application

1. Fork the GitHub repository

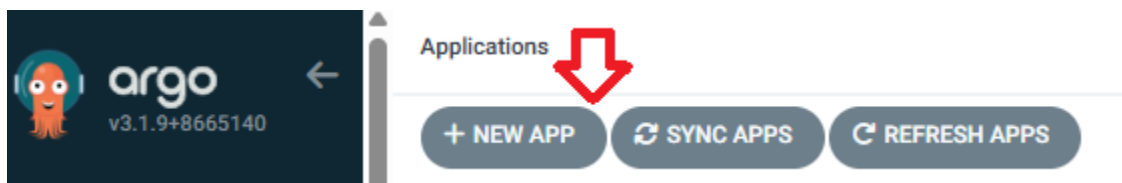
To begin, fork the following GitHub repository: <https://github.com/lfttraining/LFS256-code>. This step is crucial as it allows you to have your own copy of the application, enabling you to make and track changes independently during the lab.

Before you proceed in your fork, change the `targetPort` value in the `argocd/example-app/service.yaml` file to 80:

```
1  apiVersion: v1
2  kind: Service
3  metadata:
4    name: argocd-example-app-service
5  spec:
6    type: NodePort
7    selector:
8      app: argocd-example-app
9    ports:
10     - port: 3000
11       targetPort: 80
12     nodePort: 31000 # Optional: Kubernetes will choose a port if this is omitted
```

2. Create the application in Argo CD

To create an Application log into the UI again by accessing <https://localhost:8080> and select “+ New App”.



Start the process of creating a new application

Next,

1. Write a name of your choice in “Application Name” (e.g., “example-app”).
2. As Project Name choose “default”.
3. Under Sync Options tick “Auto-Create Namespace”.

4. In the “Source” section paste in “Repository URL” (the URL of the repository you forked in the step before).
5. In “Path” name the directory of the repository in which the YAML files are located. In this example, it will be “argocd/example-app”.
6. Under the “Destination” section define the “Cluster-URL”. In this example we use [“https://kubernetes.default.svc”](https://kubernetes.default.svc).
7. Define a namespace of your choice (e.g., “default”).
8. Select the “Create” button at the top.

CREATE
CANCEL
8

GENERAL

Application Name 1
example-app

Project Name 2
default

SYNC POLICY
Manual

☐ SET DELETION FINALIZER ⓘ

SYNC OPTIONS

☐ SKIP SCHEMA VALIDATION
☐ PRUNE LAST
☐ RESPECT IGNORE DIFFERENCES

3 ☒ AUTO-CREATE NAMESPACE
☐ APPLY OUT OF SYNC ONLY
☐ SERVER-SIDE APPLY

PRUNE PROPAGATION POLICY: foreground

☐ REPLACE ⚠️
☐ RETRY

SOURCE

Repository URL 4
https://github.com/Liquid-Reply/Argo-Training-Examples

Revision
HEAD

Path 5
example-app

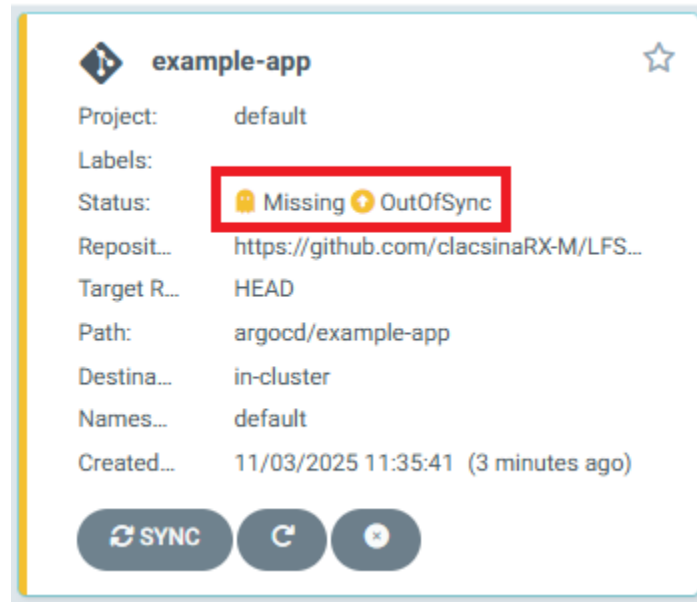
DESTINATION

Cluster URL 6
https://kubernetes.default.svc

Namespace 7
default

Process of creating a new application

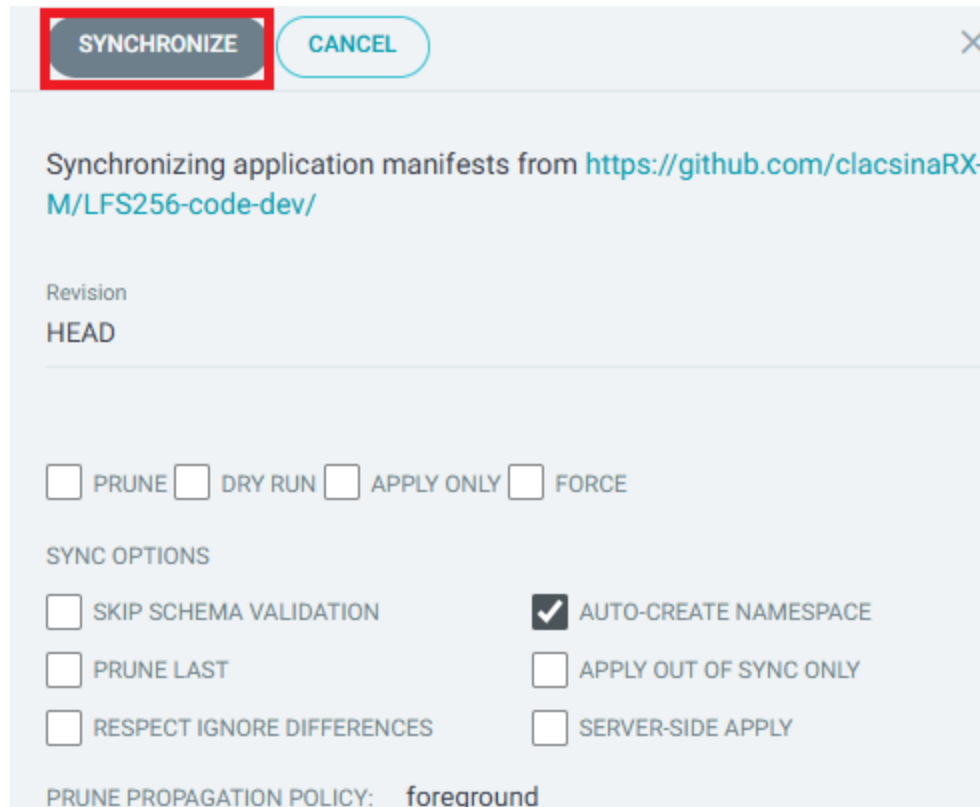
After creating the application in Argo CD, check its status using the UI. Note that the application status will be **OutOfSync** as it's yet to be deployed.



Undeployed application of Argo CD

3. Deploy the application

Select the “sync” button. This will open a new window. In that window you have several options to synchronize the application. For now we only have to choose “Synchronize”. The status should change to Synced and Progressing or Healthy.



SYNCHRONIZE **CANCEL** ✕

Synchronizing application manifests from <https://github.com/clacsinaRX-M/LFS256-code-dev/>

Revision
HEAD

☐ PRUNE ☐ DRY RUN ☐ APPLY ONLY ☐ FORCE

SYNC OPTIONS

☐ SKIP SCHEMA VALIDATION ☒ AUTO-CREATE NAMESPACE

☐ PRUNE LAST ☐ APPLY OUT OF SYNC ONLY

☐ RESPECT IGNORE DIFFERENCES ☐ SERVER-SIDE APPLY

PRUNE PROPAGATION POLICY: foreground

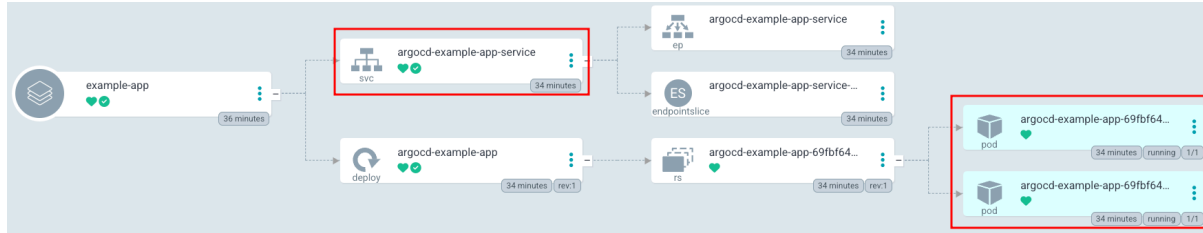
Synchronizing an application

4. Access the deployed application

Select the newly created application. In the ensuing window, you'll gain insights into its resource components, such as the two pods displayed on the right. The deployment configuration, as defined in the provided `deployment.yaml` file, specifies `replicas: 2`, indicating a desired state of two identical pods running concurrently.

Additionally, you can access logs, review events, and obtain a more detailed view of the current health status.

Make sure to take note of the associated service named `argocd-example-app-service`. This service is crucial for accessing the application through your browser. Activate port forwarding for this service to seamlessly interact with the application in your local browser environment.



Created resources for the new application

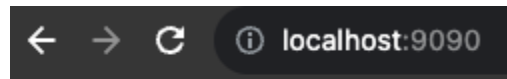
You can forward the port of the service by running the following command:

```
$ kubectl port-forward svc/argocd-example-app-service 9090:3000
--address 0.0.0.0
```

Output:

```
Forwarding from 127.0.0.1:9090 -> 3000
Forwarding from [::1]:9090 -> 3000
```

Open <http://localhost:9090> in your browser to view the application. You should see on your screen “Application Version: 1.0.0”.



Application Version: 1.0.0

Created application in the browser

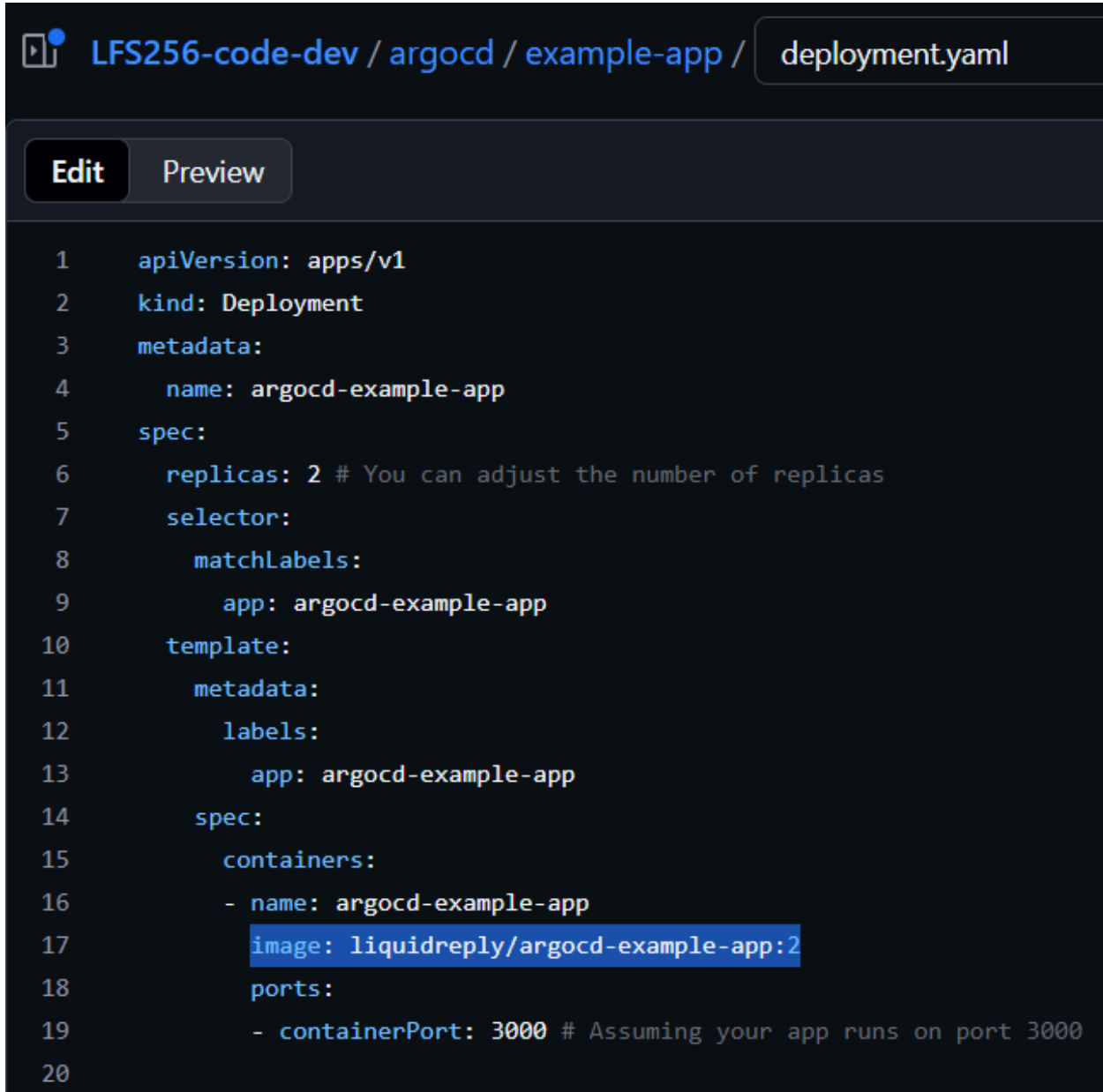
From another terminal, you can also access the application by using curl with this command:

```
$ curl localhost:9090

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <title>ArgoCD is great!</title>
</head>
<body>
  <p>Application Version: 1.0.0</p>
</body>
</html>
```

5. Update the Image

In your forked repository, navigate to the deployment YAML file (**deployment.yaml**). Locate the section where the image is specified, and change the tag to a new version. If the current image tag is `liquidreply/argocd-example-app:1`, update it to `liquidreply/argocd-example-app:2`.



```
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: argocd-example-app
5  spec:
6    replicas: 2 # You can adjust the number of replicas
7    selector:
8      matchLabels:
9        app: argocd-example-app
10   template:
11     metadata:
12       labels:
13         app: argocd-example-app
14     spec:
15       containers:
16       - name: argocd-example-app
17         image: liquidreply/argocd-example-app:2
18       ports:
19       - containerPort: 3000 # Assuming your app runs on port 3000
20
```

Commit and push the changes.

In your UI, press “Refresh”. You will see the status is now `OutOfSync`. To sync it, repeat step 4. It's also possible to activate `AutoSync`.

Choose the application. At the top left, select “App Details”. In the new window, you will find a button “Enable Auto-Sync”. Choose it and confirm it.

SUMMARYPARAMETERSMANIFESTDIFFEVENTS

EXAMPLE-APP

EDIT

PROJECTdefault

ANNOTATIONS

CLUSTERin-cluster (https://kubernetes.default.svc)

NAMESPACEdefault

CREATED AT11/03/2025 11:35:41 (9 minutes ago)

REPO URLhttps://github.com/clacsinaRX-M/LFS256-code-dev/

TARGET REVISIONHEAD

PATHargocd/example-app

SYNC OPTIONS☒ CreateNamespace

RETRY OPTIONSRetry disabled

STATUS🟡 OutOfSync from HEAD (0332952)

HEALTH🟢 Healthy

LINKS

IMAGESliquiddrepy/argocd-example-app:1

SYNC POLICY

NONE

ENABLE AUTO-SYNC

If you now change the image again or anything else in the yaml file it will automatically sync the application.

After synchronization you have to restart the port forwarding. Open <http://localhost:9090> and you should be able to see “Application Version: 2.0.0”.



Updated application in the browser

From another terminal, you can also access the application by using curl:

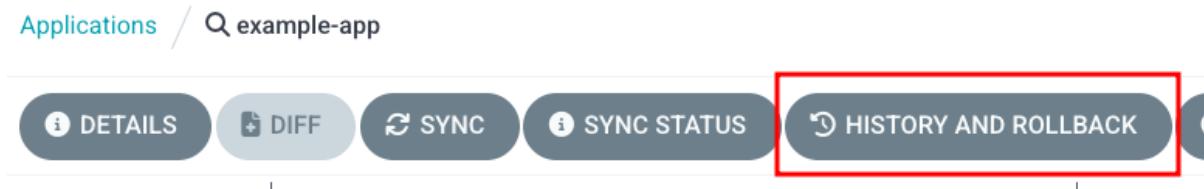
```
$ curl localhost:9090
```

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <title>ArgoCD is great!</title>
</head>
<body>
  <p>Application Version: 2.0.0</p>
</body>
</html>
```

6. Rollback

Now, let's delve into Argo CD's rollback capability. After updating your application, we'll guide you through effortlessly reverting to the previous version.

In the UI, you will find inside the application a button labeled “History and Rollback”.



Menu of an application

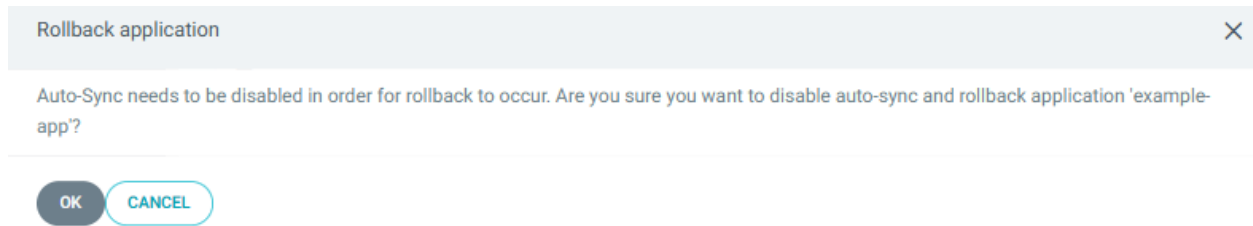
If you select it, a new window will open. In this window, you can see the history of deployment for your application.

 Deployed At: a minute ago (Mon Nov 03 2025 11:45:43 GMT-0800)  Time to deploy: 1s Initiated by: automated sync policy Active for: 1m8s	Revision: 0332952  Authored by Christian Alday Lacsina <43353000+clacsinaRX-M@users.noreply.github.com> 3 minutes ago (Mon Nov 03 2025 11:43:56 GMT-0800) Update container image version to 2 GPG signature - Source Parameters URL: https://github.com/clacsinaRX-M/LFS256-code-dev/
 Deployed At: 7 minutes ago (Mon Nov 03 2025 11:39:51 GMT-0800)  Time to deploy: 0s Initiated by: admin Active for: 5m52s	Revision: dc12ded  Authored by Christian Alday Lacsina <43353000+clacsinaRX-M@users.noreply.github.com> 15 minutes ago (Mon Nov 03 2025 11:32:10 GMT-0800) Change targetPort from 3000 to 80 in service.yaml GPG signature - Source Parameters URL: https://github.com/clacsinaRX-M/LFS256-code-dev/

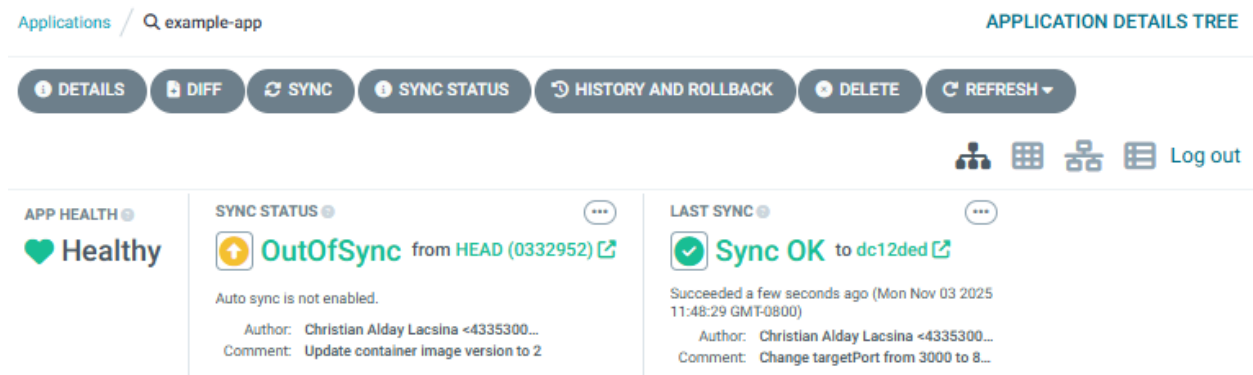
Besides information about the deployment itself, you will also be able to see a Kebab menu.

 Deployed At: 8 minutes ago (Mon Nov 03 2025 11:39:51 GMT-0800)  Time to deploy: 0s Initiated by: admin Active for: 5m52s	Revision: dc12ded  Authored by Christian Alday Lacsina <43353000+clacsinaRX-M@users.noreply.github.com> 15 minutes ago (Mon Nov 03 2025 11:32:10 GMT-0800) Change targetPort from 3000 to 80 in service.yaml GPG signature - Source Parameters URL: https://github.com/clacsinaRX-M/LFS256-code-dev/	 Rollback
---	---	---

Select it and choose to rollback to that deployment. If you have Autosync on, you will be prompted to turn it off for the rollback to occur:



After a few moments, the app will report that it is out of sync:



Redo the kubectl port-forward and try accessing your app again:

```
$ kubectl port-forward svc/argocd-example-app-service 9090:3000
--address 0.0.0.0
```

```
$ # in another terminal
```

```
$ curl localhost:9090
```

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
```

```
  <title>ArgoCD is great!</title>
```

```
</head>

<body>

    <p>Application Version: 1.0.0</p>

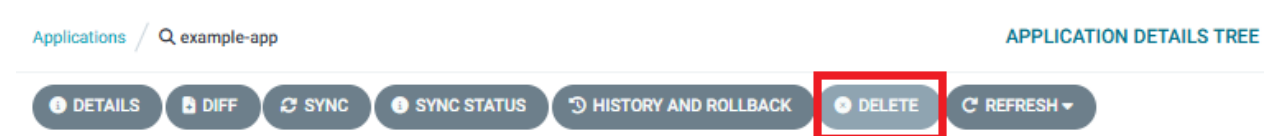
</body>

</html>
```

7. Deleting An Application

At some point, you will have to retire an active application. Argo CD makes this easy by tying all of the assets of an app in Kubernetes (e.g. the deployments, services, etc.) to the existence of its associated Application custom resource.

In your UI, click “Delete” in the application’s toolbar:



You will be prompted to confirm the deletion.

It describes some of the things to consider, such as the deletion policy.

There are 3 policies available:

- Foreground propagation - which has Argo CD terminate all child resources before the parent.
- Background propagation - which has ArgoCD delete the parent first and lets Kubernetes remove the child objects asynchronously as kubectl normally would.
- Do not touch the resources at all - which will keep the Kubernetes resources but remove it from Argo CD’s management.

Delete application

×

!

Are you sure you want to delete the **Application** `example-app`?

Deleting the application in `foreground` or `background` mode will delete all the application's managed resources, which can be **dangerous**. Be sure you understand the effects of deleting this resource before continuing. Consider asking someone to review the change first.

Please type '`example-app`' to confirm the deletion of the resource

Select propagation policy for application deletion

☒ Foreground ⓘ ☐ Background ⓘ ☐ Non-cascading ⓘ

Type your app's name and click OK with the foreground policy enabled.

